

– поле *cmd* задает команду запуска сервера языковой поддержки.

Важной частью конфигурации является поле *verdictClassName*. Это поле связывает декларативное описание языка с плагиновой частью системы. При выполнении проверки сервис *test-runner* запускает тестовый фреймворк в контейнере, получает отчет и код завершения процесса, после чего должен привести результат к единой модели. Такая унификация необходима потому, что разные тестовые фреймворки по-разному описывают ошибки, пропущенные тесты, имена наборов тестов и длительность выполнения. Даже когда несколько инструментов формируют отчет в формате «*junit.xml*», структура элементов, атрибуты и смысл отдельных полей могут немного отличаться. Кроме того, архитектура не должна зависеть только от «*junit.xml*», так как новый язык или новый фреймворк может возвращать результат в другом формате.

Именно для этой задачи используется языковой плагин. Плагин получает текст отчета и код завершения процесса, анализирует их с учетом особенностей конкретного тестового фреймворка и возвращает внутренний формат результата. Для Python таким плагином является класс «*org.supremecode.commonplugins.PyTestLanguageTester*». Его имя записывается в *verdictClassName*, поэтому ядро *test-runner* не выбирает обработчик по набору условных операторов и не содержит знания о формате отчета PyTest. Ядро знает только общий контракт, а конкретная логика разбора находится в подключаемой реализации.

Контракт для языковых плагинов расположен в модуле *plugin-sdk*. Этот модуль можно рассматривать как границу между стабильной частью платформы и расширениями для отдельных языков. В нем определен интерфейс *LanguageTester*, который содержит функцию *verdict*. Эта функция принимает отчет тестового фреймворка и код завершения процесса, а возвращает объект *TestExecutionResult*. Следовательно, любой языковой плагин обязан реализовать единый способ получения вердикта, но внутри реализации может использовать собственный алгоритм разбора отчета. Диаграмма классов, связанная с результатами тестированием представлена на рисунке 2.4.

Подключение через интерфейс позволяет изолировать изменения, связанные с поддержкой новых языков программирования и тестовых инструментов. При расширении платформы требуется реализовать новый языковой плагин и указать его класс в конфигурации, не изменяя код ядра *test-runner* и существующие механизмы обработки результатов.

- поддержка LSP, todo-списков, веб-запросов и вопросов к пользователю;
- настраиваемые правила в agents.md;
- agentic workflow (многошаговые рассуждения и действия);
- продвинутая работа с контекстом;
- параллельный вызов инструментов;
- коннекторы к различным провайдерам и моделям;
- MCP (Model Context Protocol);
- Skills (навыки);
- RAG (Retrieval-Augmented Generation);
- субагенты.

Этот перечень постоянно расширяется, и разработчики кодинг-агентов активно добавляют новый функционал. В сфере программирования разумным подходом является переиспользование готовых возможностей, а не их повторная реализация.

Предлагаемое решение представлено в виде интеграции фронтенда платформы с локальным MCP-сервером [12], который, в свою очередь, взаимодействует с кодинг-агентом. Фронтенд предоставляет MCP-серверу набор функций по протоколу WebSocket. К числу таких функций относятся:

- *platformInfo* (информация о платформе);
- *exportThisProblem* (экспорт текущей задачи);
- *importThisProblem* (импорт задачи);
- *validate* (валидация задачи);
- *listProblems* (список существующих задач);
- *exportProblem* (экспорт произвольной задачи).

Локальный MCP-сервер предоставляет эти же функции кодинг-агенту уже по протоколу MCP. Схема взаимодействия кодинг-агента с фронтендом через MCP-сервер представлена на рисунке 2.5.

Такой подход позволяет сосредоточиться на предметной логике генерации задач и использовании возможностей платформы, передав кодинг-агенту функции анализа, планирования и работы с контекстом. В результате расширение системы достигается за счет интеграции существующих инструментов, а не разработки полноценной агентной среды внутри SupremeCode.

- схема данных файла «info.json» и обязательность указания названия задачи и списка тегов;
- особенности тестирования для поддерживаемых языков программирования;
- порядок получения актуальной информации о платформе через инструмент *platform-info*;
- порядок импорта, экспорта и проверки задачи через инструменты *importThisProblem*, *exportThisProblem* и *validate*.

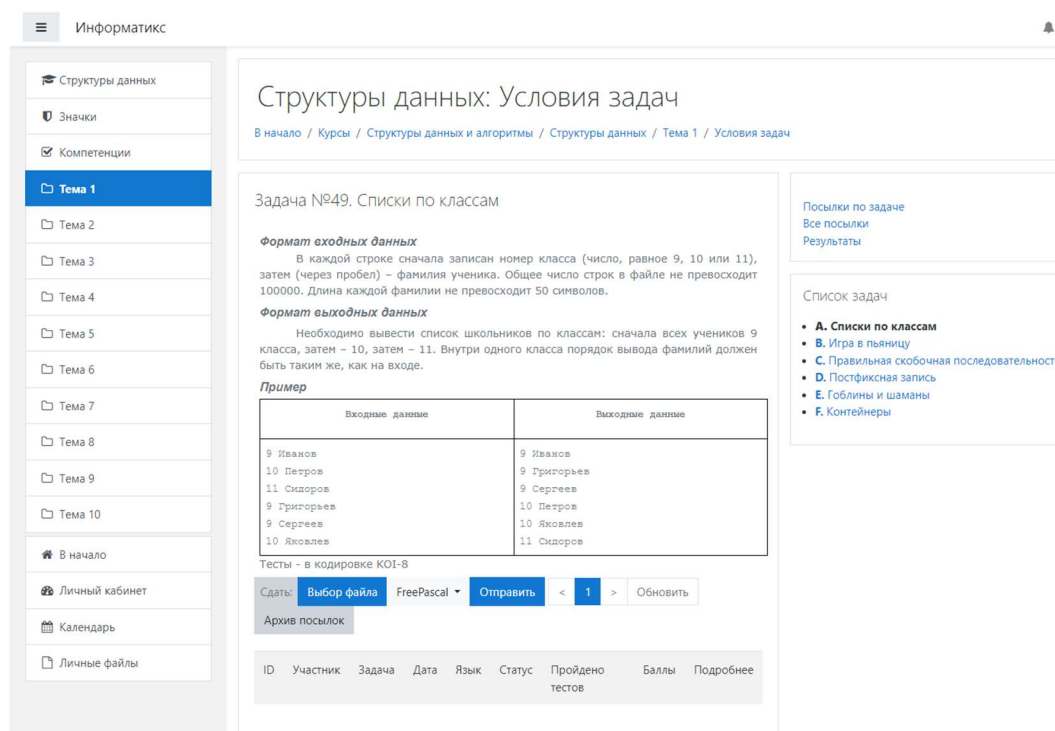
Файл «SKILL.md» выступает формальным описанием правил, по которым ИИ-агент должен анализировать, редактировать и проверять задачу. При необходимости пользователь может доработать этот файл, чтобы уточнить требования к поведению агента и повысить воспроизводимость редактирования задач. При необходимости пользователь может самостоятельно дорабатывать навык, если не получает от агента ожидаемого результата.

После этого пользователь начинает обсуждать с кодинг-агентом задачу по программированию. Агент определяет предметную область (домен) и считывает скилл, связанный с созданием задачи. Далее агент работает с локальными файлами, запускает экспорт подготовленных файлов обратно на фронтенд и инициирует валидацию задачи. Пользователь сохраняет задачу, если результат его устраивает. В случае, если агенту не хватает примеров для качественной генерации, он может получить список уже созданных пользователем задач, скачать эти задачи и использовать их в качестве референсов для улучшения качества. Пользователь может изменить навык таким образом, чтобы агент всегда скачивал определённые референсы задач.

Итоговая схема компонента AI генерации задач представлена на рисунке 2.6.

Предложенная схема сохраняет контроль над содержимым задачи за пользователем, но при этом переносит значительную часть рутинной работы на кодинг-агента. Использование навыков и инструментов платформы обеспечивает единый процесс создания, проверки и последующего сопровождения задач независимо от выбранного агентного решения. Благодаря этому взаимодействие между платформой и агентом строится на формализованных правилах и стандартизированных интерфейсах, что повышает воспроизводимость результатов и упрощает дальнейшее развитие механизма генерации задач.

С позиции пользовательского интерфейса «Информатикс» заметно уступает современным стандартам: оформление платформы морально устарело, а главная страница перегружена информационными блоками, что затрудняет навигацию. Поиск задач дополнительно осложняется их числовой идентификацией, поскольку соответствующие номера, как правило, известны лишь преподавателям. Интерфейс страницы задачи представлен на рисунке 1.1.



Платформа «LeetCode», в свою очередь, отличается современным визуальным оформлением и продуманной структурой каталога задач. Широкие возможности фильтрации по тематике, сложности и тегам существенно облегчают навигацию и поиск нужного материала, что положительно сказывается на пользовательском опыте в целом. Внешний вид страницы задачи проиллюстрирован на рисунке 1.2.

Платформа «Codewars» также располагает удобным интерфейсом, рассчитанным на разработчиков с разным уровнем подготовки. Среди ключевых возможностей — гибкий поиск задач по тематике, уровню сложности и языку программирования. Отдельного внимания заслуживает инструментарий для взаимодействия с сообществом: пользователи могут

обсуждать условия задач и варианты решений, что способствует более глубокому усвоению материала и обмену практическим опытом между участниками. Внешний вид страницы задачи проиллюстрирован на рисунке 1.3.

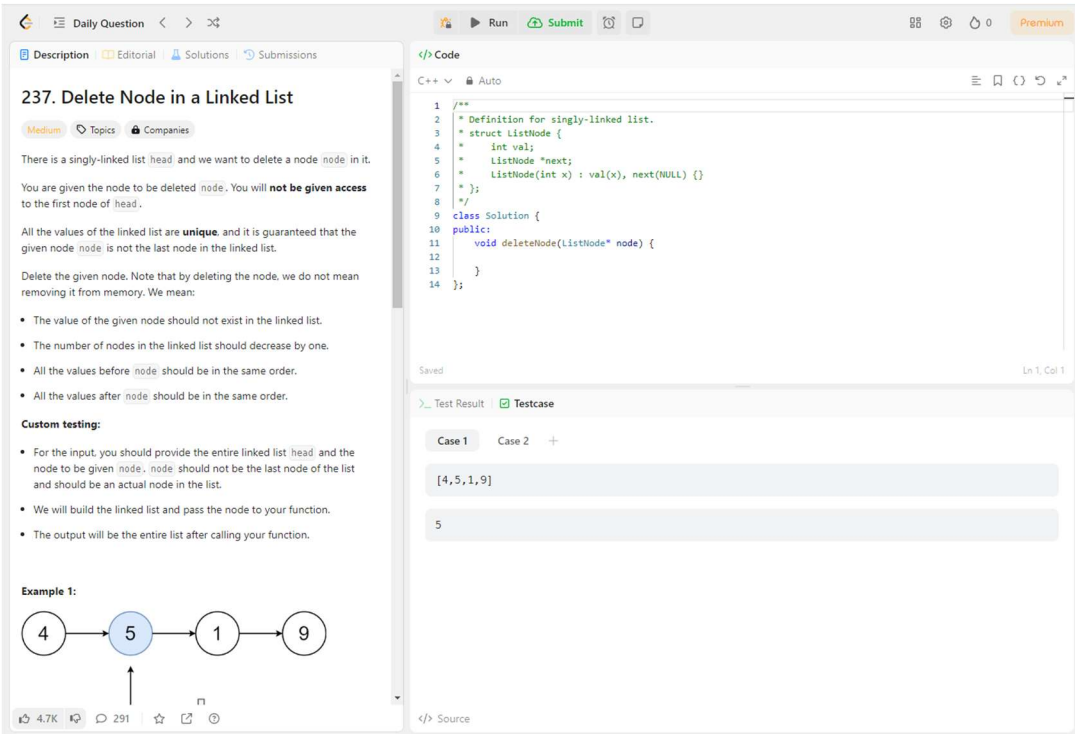


Рисунок 1.2 – Платформа «LeetCode»

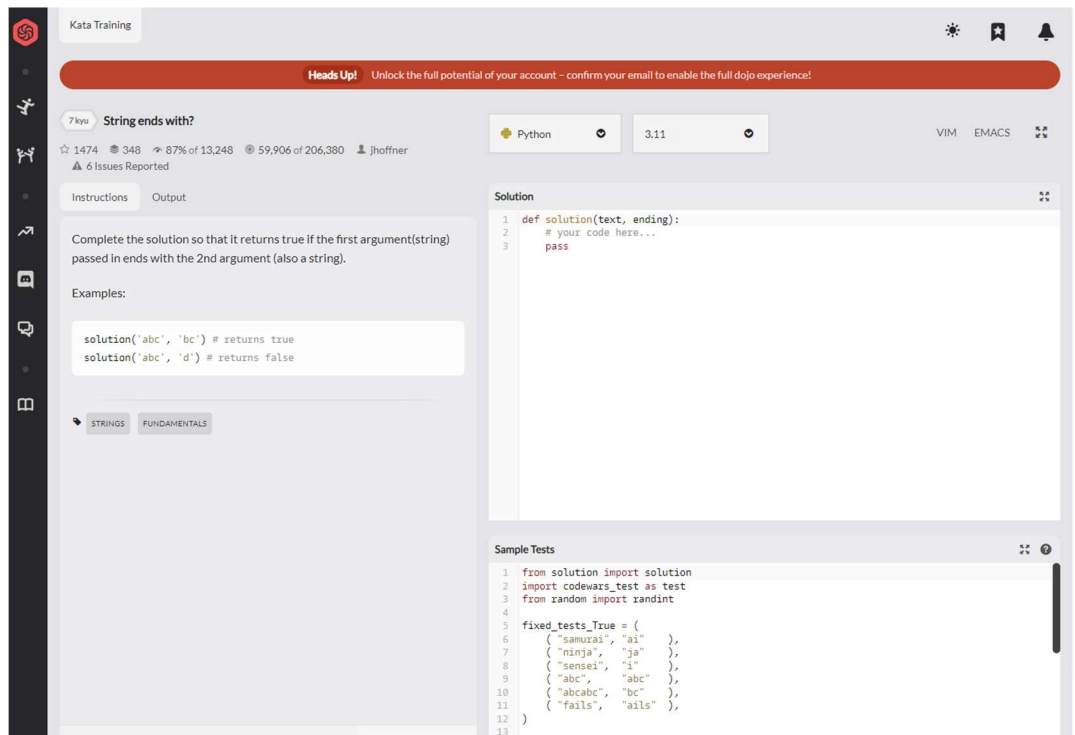


Рисунок 1.3 – Платформа «CodeWars»

Платформа «HackerRank» предоставляет понятный и логично организованный интерфейс, ориентированный на широкую аудиторию — от начинающих до опытных разработчиков. Задачи систематизированы в структурированный каталог с возможностью фильтрации по тематике, уровню сложности и языку программирования.

Страница каждой задачи содержит развёрнутое описание условия, спецификацию входных и выходных данных, а также разобранные примеры. Встроенный редактор с подсветкой синтаксиса и поддержкой запуска кода непосредственно на платформе позволяет проверять решения без привлечения внешних инструментов.

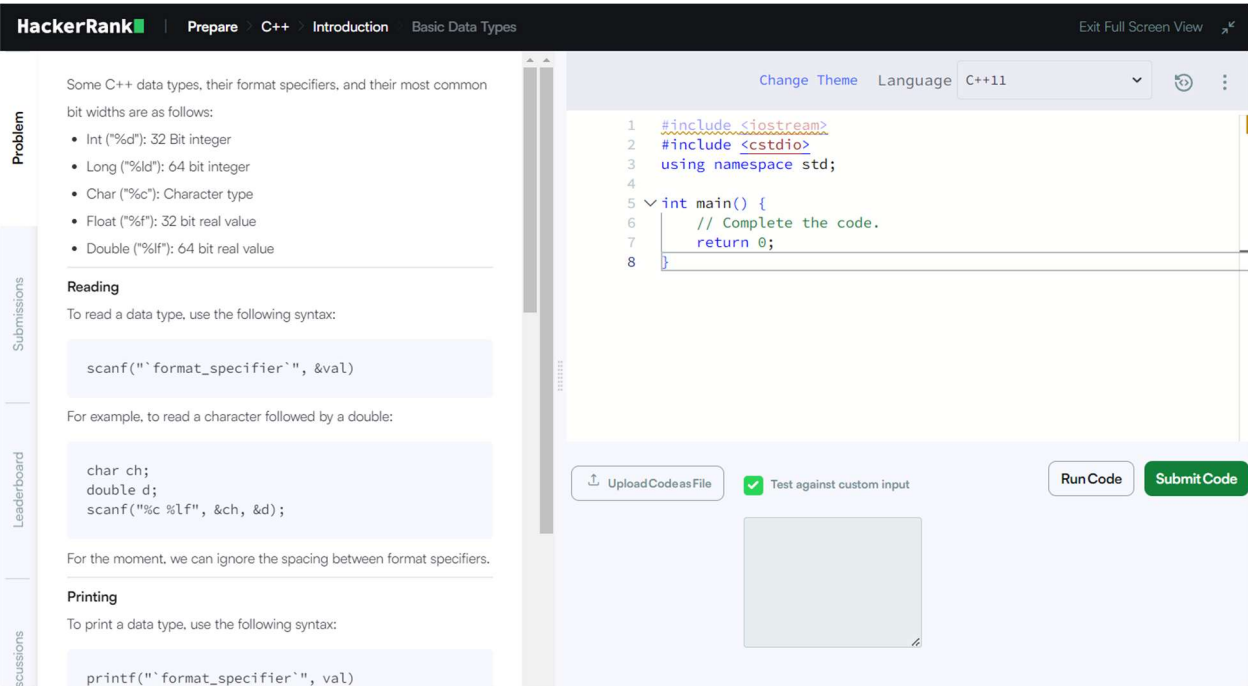


Рисунок 1.4 – Платформа «HackerRank»

Среди рассматриваемых платформ «LeetCode» выделяется наличием интерактивной среды для выполнения кода — «песочницы» (*playground*), функционирующей независимо от системы задач. Данный инструмент открывает разработчикам пространство для свободного экспериментирования с кодом, что благоприятно влияет на развитие профессиональных компетенций.

Подобная среда даёт возможность проверять, отлаживать и совершенствовать алгоритмы вне контекста конкретных задач. Пользователи могут реализовывать собственные идеи, осваивать новые концепции и подходы к разработке, а также изучать различные языки программирования без каких-либо ограничений.

- `shared`: общая библиотека утилит и моделей данных, используемых `web` и `test-runner`.
- `plugin-sdk`: набор интерфейсов и базовых классов для создания подключаемых модулей.
- `common-plugins`: коллекция стандартных реализаций плагинов, поставляемых вместе с платформой.

3.3.1 Описание модуля «web»

Модуль `web` отвечает за взаимодействие с пользователем. Он состоит из нескольких пакетов: *playground*, *problem*, *user*.

Начнем с описания пакета *problem*. В нем содержатся классы по работе с задачами и их решениями. На рисунке 3.2 представлена диаграмма классов пакета *problem*.

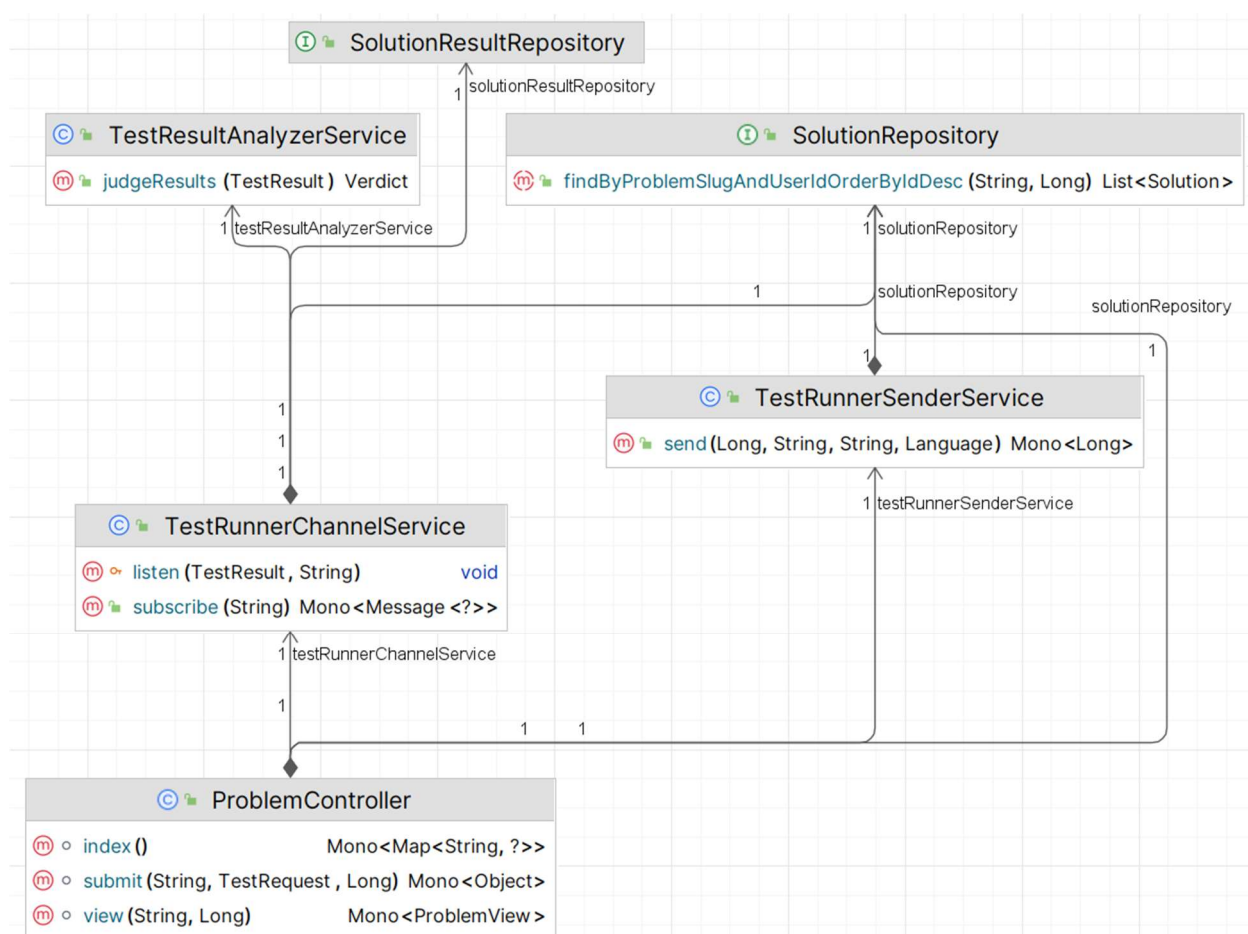


Рисунок 3.2 – Диаграмма классов пакета *problem*



Рисунок 3.3 – Диаграмма классов модуля test-runner

Класс *WaitCallback* – *callback* для получения кода завершения контейнера.

Класс *SetTimeout* – создаёт *CompletableFuture*, который завершается через заданное время.

Таким образом, иерархия классов позволяет абстрагировать логику тестирования кода на различных языках программирования и обеспечивает гибкость и расширяемость системы.

3.3.3 Описание модуля task-runner

Модуль *task-runner* отвечает за выполнение кода пользовательских программ. Он получает программу от модуля *web*, создает изолированный

Класс *LanguageConfig* — конфигурационный POJO-объект для хранения параметров языка: образа Docker, команды запуска и имени файла.

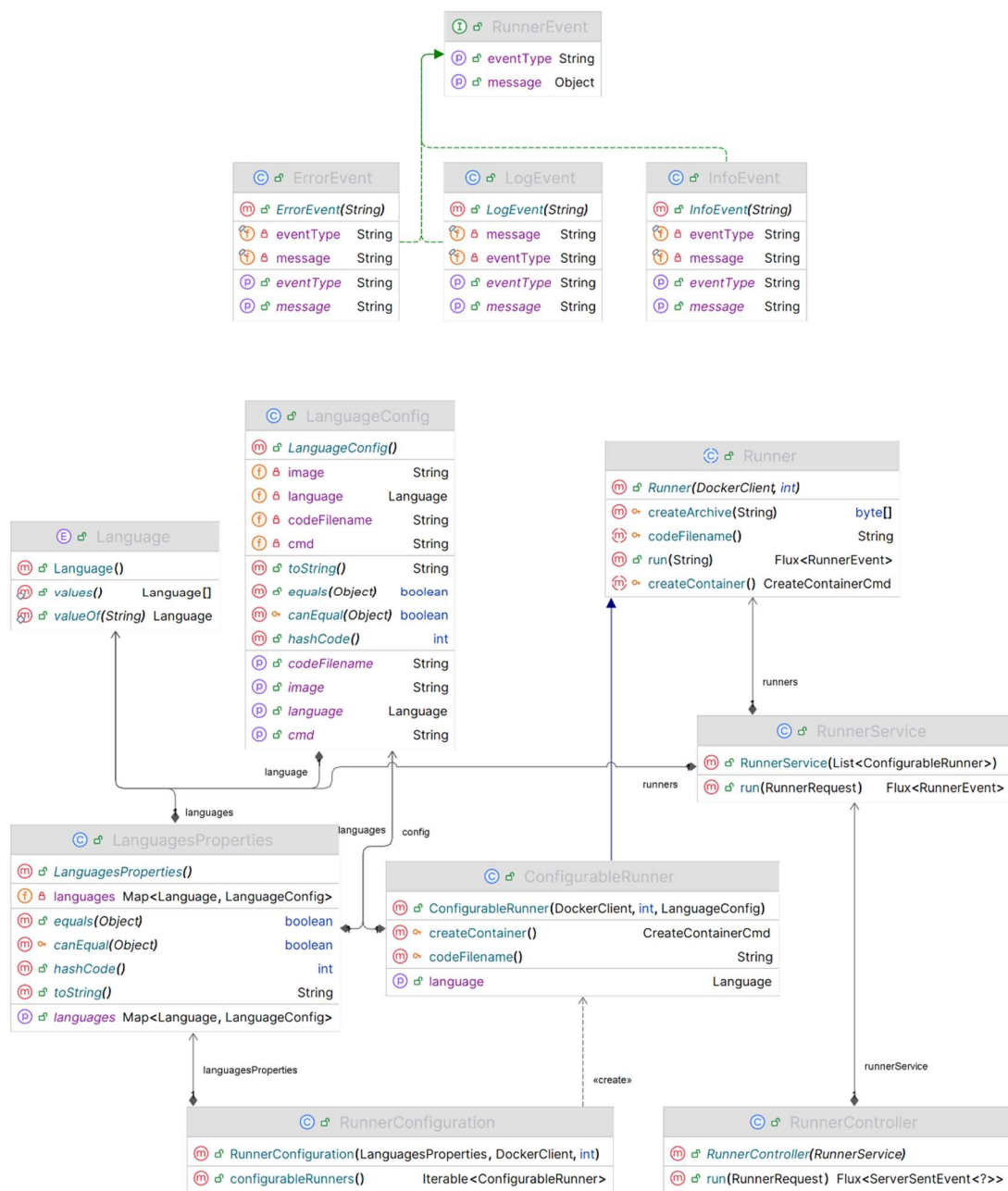


Рисунок 3.4 – Диаграмма классов модуля task-runner

3.4 Конструирование SDK для разработки языковых плагинов

Программный SDK для языковых плагинов строится на трёх взаимосвязанных модулях: plugin-sdk, common-plugins и test-runner. Каждый

Кликнув на любую задачу, откроется страница решения задачи, она представлена на рисунке 4.4. На странице есть подробное описание задачи, примеры входных и выходных данных, ограничения по входным данным. Можно выбрать один из трех языков программирования.

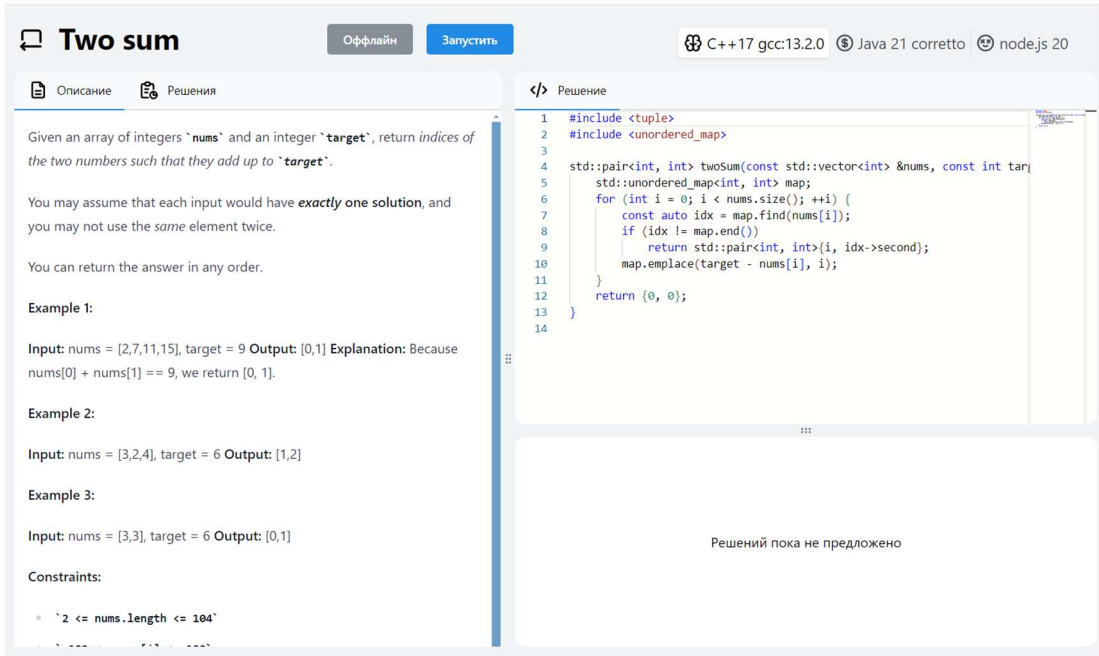


Рисунок 4.4 – Страница решения задачи

Когда напишете код, решающий задачу, и будете готовы к проверке решения, нажмите кнопку «Запустить». Пока решение еще не протестировано, отображается спиннер загрузки. Перезагружать страницу самостоятельно не требуется. Результаты автоматически отобразятся на страницу без полной перезагрузки. Дождавшись завершения тестирования, можно увидеть на рисунке 4.5, что решение было одобрено. В левой панели отображается таблица с результатами решений. Также отображается дополнительная информация со статусом завершения программы, количеством выполненных тестов, ошибок, сбоев и временем выполнения. Как итог, задача решена. Справа от названия задачи отображается зеленая галочка, обозначающая успешное решение задачи.

Если вам нужна площадка для программирования вне решения задачи, например, для проверки как работают стандартные функции вашего языка, то можете перейти на страницу «компилятора», она представлена на рисунке 4.6. На ней все также есть кнопка для запуска программы. Можно заметить, что текстовые сообщения о результатах выполнения программы появляются в правой панели по мере выполнения программы, а не после завершения. Это сложно передать скриншотом в отчете, но в реальности оно так и работает.